

PORTADA

# Calidad de Software

## Semana 15 — Integración al pipeline y pruebas inestables

Universidad de Antioquía · Ingeniería de Sistemas



REPASO RELÁMPAGO

## Lo que ya tenemos de las semanas 13 y 14

- Un **framework de automatización** (Selenium, Playwright o Cypress) con un primer test E2E, y un **pipeline de CI** desde la semana 12 (build, test, cobertura, análisis estático).
- Una suite **refactorizada con Page Object**: los selectores viven en una clase por pantalla, no repetidos en cada test.
- Lo que falta: esa suite todavía se ejecuta **a mano**, no en cada `commit`. Y nadie ha preguntado qué pasa si un test falla sin que nada haya cambiado.



APERTURA

# **84 % de las pruebas que cambian de verde a rojo en Google no señalan un bug**

Son inestables. El código no cambió — la prueba sí. John Micco, Google, 2016.

CASO REAL · FICHA

# El estudio de Google sobre pruebas inestables

| Dato     | Detalle                                                                                                                                             |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Quién    | John Micco, equipo de infraestructura de pruebas de Google.                                                                                         |
| Cuándo   | Google Test Automation Conference (GTAC), 2016.                                                                                                     |
| Escala   | Cientos de miles de pruebas automatizadas, millones de ejecuciones diarias sobre el monorepo interno de Google.                                     |
| Hallazgo | ~1,5 % de todas las ejecuciones de prueba mostraban un resultado inestable — a veces verde, a veces rojo, sin ningún cambio de código de por medio. |



CASO REAL · QUÉ HIZO GOOGLE

## Un 1,5 % que cuesta miles de horas de ingeniería

El número parece pequeño, pero a esa escala se traduce en **cientos de miles de ejecuciones inestables por día**. Cada una obliga a alguien a detenerse, revisar un fallo que no era real y decidir si reintentar o investigar. Google respondió con tres medidas, no con una sola:

- Un **puntaje de inestabilidad (flakiness score)** por prueba, calculado con el historial de ejecuciones.
- **Reintento automático** de pruebas con puntaje alto antes de marcar el build como roto.
- **Desactivación automática** de pruebas que superan un umbral de inestabilidad, con alerta al equipo dueño.



## TRANSICIÓN

# Hoy: conectar la suite al pipeline, y aprender a convivir con la inestabilidad

No hace falta la escala de Google para toparse con este problema — una suite de diez pruebas E2E ya puede tener una inestable. Hoy resolvemos dos cosas: **cómo** se integra la suite automatizada al pipeline de CI de la semana 12, y **qué hacer** cuando una prueba de esa suite empieza a fallar sin razón aparente.



## CONCEPTO

# Qué es una prueba inestable (flaky test)

Es una prueba automatizada que produce **resultados distintos en ejecuciones distintas, sobre el mismo código**: a veces pasa, a veces falla, sin que nadie haya tocado ni la aplicación ni la prueba. No es un test que falla siempre por un bug real — ese es un test **útil**, aunque incómodo.

El problema de fondo no es que falle. Es que **deja de ser una señal confiable**: si un test falla y el 84 % de las veces es ruido, el equipo aprende a ignorar los fallos — incluido el día en que el fallo sí es un bug real.



CONCEPTO

# Causas comunes de inestabilidad

| Causa                                | Qué pasa                                                                                                    |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Esperas fijas ( <code>sleep</code> ) | La prueba asume que un elemento carga en X segundos; si la red va más lenta ese día, falla.                 |
| Estado compartido entre pruebas      | Un test deja datos que otro test asume que no existen; el orden de ejecución cambia el resultado.           |
| Dependencias externas                | La prueba llama una API de terceros, un servicio de correo o un reloj del sistema fuera de nuestro control. |
| Concurrencia                         | Dos operaciones asíncronas terminan en distinto orden según la carga del entorno de ejecución.              |
| Datos de prueba no deterministas     | La prueba genera un valor aleatorio o depende de la fecha del día sin fijarlos.                             |



```
table
  causa      |      que_pasa
  ---|---
  Esperas fijas ( sleep ) | La prueba asume que un elemento carga en X segundos; si la red va más lenta ese día, falla.
  Estado compartido entre pruebas | Un test deja datos que otro test asume que no existen; el orden de ejecución cambia el resultado.
  Dependencias externas | La prueba llama una API de terceros, un servicio de correo o un reloj del sistema fuera de nuestro control.
  Concurrencia | Dos operaciones asíncronas terminan en distinto orden según la carga del entorno de ejecución.
  Datos de prueba no deterministas | La prueba genera un valor aleatorio o depende de la fecha del día sin fijarlos.
```

ANTES

## Espera fija: funciona en la máquina del desarrollador, falla en CI

```
def test_agregar_al_carrito(page):  
    page.click("#add-to-cart-sauce-labs-backpack")  
    time.sleep(1) # "seguro que ya cargó"  
    assert page.locator(".shopping_cart_badge").text_content() == "1"
```

En la laptop del desarrollador, un segundo alcanza. En el runner de CI, compartido y más lento en horas pico, a veces no. El test no prueba la aplicación — prueba qué tan rápido estuvo el servidor de CI esa ejecución.

DESPUÉS

## Espera explícita: condicionada al estado, no al reloj

```
def test_agregar_al_carrito(page):  
    page.click("#add-to-cart-sauce-labs-backpack")  
    page.wait_for_selector(".shopping_cart_badge", timeout=5000)  
    assert page.locator(".shopping_cart_badge").text_content() == "1"
```

La prueba espera **hasta que el elemento exista**, con un tope máximo razonable — no un tiempo fijo adivinado. Si el badge tarda 200 ms o 3 segundos, la prueba sigue siendo determinista frente al mismo resultado final.

## CONCEPTO

# Aislamiento de pruebas (test isolation)

Cada prueba debe poder ejecutarse **sola, en cualquier orden, y llegar al mismo resultado**. Eso exige que no dependa de datos que dejó otra prueba, ni de que una prueba anterior haya corrido primero.

En la práctica: cada prueba prepara su propio estado inicial (con una *fixture*, semana 14) y no asume nada sobre lo que quedó de la ejecución anterior. Si dos pruebas comparten el mismo usuario o el mismo carrito sin aislarse, una modifica lo que la otra necesita intacto.



CONTRAEJEMPLO

## Dos pruebas que solo pasan juntas, en orden

```
def test_1_login(page):  
    LoginPage(page).login("standard_user", "secret_sauce")  
    # deja la sesión abierta para el siguiente test  
  
def test_2_agregar_al_carrito(page):  
    # asume que test_1_login ya corrió antes en esta misma página  
    page.click("#add-to-cart-sauce-labs-backpack")
```

Si `test_2` corre solo, o el framework decide ejecutar las pruebas en paralelo o en otro orden, falla — no porque la aplicación tenga un defecto, sino porque la prueba nunca declaró su propia dependencia. Cada prueba debe iniciar sesión por su cuenta.

## TRANSICIÓN

# Con una suite más confiable, toca conectarla al pipeline

El pipeline de la semana 12 ya ejecuta pruebas unitarias en cada `commit`. Una suite E2E es distinta: más lenta, necesita un navegador y no puede correr en el mismo paso que el build. Se agrega como una **etapa adicional**, no como un reemplazo.



CONCEPTO

# Qué cambia al integrar E2E al pipeline

|              | Pruebas unitarias (semana 12)         | Suite E2E (hoy)                                                                           |
|--------------|---------------------------------------|-------------------------------------------------------------------------------------------|
| Duración     | Segundos                              | Minutos — abre navegador real o headless                                                  |
| Entorno      | Solo el código                        | Necesita la aplicación desplegada o corriendo localmente                                  |
| Cuándo corre | En cada <code>commit</code> , siempre | En cada <code>commit</code> , en una etapa aparte y en paralelo si el pipeline lo permite |
| Si falla     | Bloquea el build casi siempre         | Bloquea, salvo pruebas en cuarentena (ver más adelante)                                   |



EJEMPLO

## La suite E2E como job separado en GitHub Actions

```
jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - run: pytest tests/unit

  e2e-tests:
    runs-on: ubuntu-latest
    needs: unit-tests
    steps:
      - run: playwright install --with-deps
      - run: pytest tests/e2e --browser=chromium --headless
```

`e2e-tests` solo arranca si `unit-tests` pasó primero ( `needs` ), y corre el navegador en modo `headless` — sin interfaz gráfica, como necesita un runner de CI.

## CONCEPTO

# Qué hacer con una prueba que ya se sabe inestable

- **Arreglar la causa raíz** (esperas explícitas, aislamiento) — siempre la primera opción.
- **Reintento automático (retry)**: repetir la prueba una o dos veces antes de marcarla como fallida. Mitiga el ruido, no arregla la causa.
- **Cuarentena (quarantine)**: sacar la prueba del conjunto que bloquea el pipeline, pero seguir ejecutándola y reportando su resultado por separado, hasta corregirla.

La cuarentena no es "borrar el test" ni "ignorarlos para siempre": es una casilla temporal con dueño y fecha de revisión.



TENSIÓN CONCEPTUAL

## El reintento automático, ¿ayuda o esconde el problema?

Configurar reintentos es rápido: una línea en el pipeline y los builds rojos por ruido bajan de inmediato. El equipo respira. Pero un reintento que siempre pasa a la segunda vez **nunca vuelve a la lista de pendientes** — la causa raíz queda enterrada bajo un mecanismo que la disimula, y la suite se vuelve cada vez más lenta porque cada prueba corre dos o tres veces.

**El reintento es una mitigación temporal con fecha de vencimiento, no una política permanente.** Cada prueba en retry automático debería tener asignado quién la va a arreglar y para cuándo — igual que la cuarentena.



CONCEPTO

# El costo de mantenimiento de una suite automatizada

Escribir la suite es solo el primer costo. Después de eso, cada semana que la aplicación cambia, alguien tiene que:

| Actividad                     | Por qué ocurre                                                                               |
|-------------------------------|----------------------------------------------------------------------------------------------|
| Actualizar selectores         | La interfaz cambia — mitigado, no eliminado, por Page Object (semana 14).                    |
| Investigar fallos inestables  | Distinguir un bug real de ruido consume tiempo de ingeniería en cada ejecución roja.         |
| Revisar la cola de cuarentena | Pruebas aparcadas que nadie revisa se acumulan y dejan de probar nada.                       |
| Ajustar tiempos de ejecución  | Una suite que tarda 40 minutos frena cada <code>commit</code> — hay que paralelizar o podar. |



INTERACCIÓN

## Ejercicio rápido: ¿cuál es la causa?

**5 minutos, respondan en el chat:**

1. Una prueba de checkout falla solo los lunes, cuando el equipo de datos recarga el catálogo de productos de prueba.
2. Una prueba pasa siempre que se ejecuta sola, pero falla si corre después de la prueba de "eliminar cuenta".
3. Una prueba que verifica una notificación falla una de cada cinco veces, sin patrón aparente de horario ni de datos.

CONCEPTO EN DISPUTA

## ¿Más pruebas automatizadas es siempre mejor?

La semana 11 vio la cobertura como métrica: alta cobertura no garantiza calidad, solo dice qué porcentaje del código se ejecutó. Hoy se suma un segundo matiz: una suite grande y **inestable** puede ser peor que una suite pequeña y confiable — porque erosiona la confianza del equipo en el pipeline completo, no solo en esa prueba.

La pregunta correcta no es "¿cuántas pruebas E2E tenemos?", sino "¿cuántas de nuestras pruebas E2E el equipo revisa de verdad cuando fallan?". Una suite que se ignora no está probando nada, aunque aparezca verde.



## SÍNTESIS

# Lo que hay que llevarse de hoy

1. Una **prueba inestable** no falla por un bug: falla de forma distinta sobre el mismo código, y erosiona la confianza en toda la suite.
2. Sus causas más comunes: **esperas fijas, estado compartido, dependencias externas, concurrencia y datos no deterministas**.
3. **Esperas explícitas** y **aislamiento de pruebas** son las correcciones de raíz, no atajos.
4. La suite E2E se integra al pipeline de la semana 12 como **etapa aparte**, más lenta y con su propio entorno.
5. **Reintento y cuarentena** son mitigaciones temporales, con dueño y fecha — no una forma de esconder el problema.
6. El costo de una suite automatizada no termina al escribirla: **mantenerla es el costo mayor**.



## INSTRUCCIONES

# Taller de hoy: la suite, integrada al pipeline

2 horas, en sala de Zoom asignada — equipo fijo de Fábrica-Escuela, sobre SauceDemo

- Construyan (o retomen si ya la tienen) una suite E2E sobre **SauceDemo**, con la credencial asignada a su equipo, aplicando Page Object (semana 14) y aislamiento de pruebas (hoy): login, agregar producto al carrito y checkout, como mínimo.
- Agreguen la suite al **pipeline de CI** que su equipo construyó en la semana 12, como una etapa separada de las pruebas unitarias, ejecutando el navegador en modo *headless*.
- La suite debe quedar **ejecutándose automáticamente en cada commit** — no a mano.



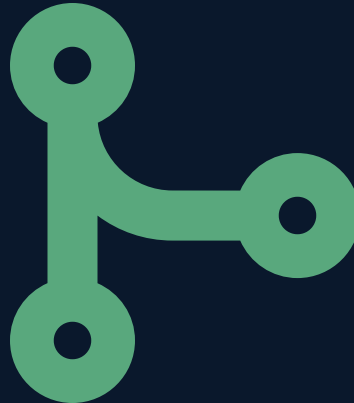
## INSTRUCCIONES

# Entregable: pipeline en verde y reporte de inestabilidad

Además del pipeline operativo, el equipo entrega un documento breve con:

- **Evidencia** de al menos una ejecución del pipeline disparada por un `commit` real, con la etapa E2E visible y en verde.
- Un reporte de **si encontraron algún comportamiento inestable** durante el taller (una prueba que falló sin motivo aparente) y qué causa de la sección de hoy explica mejor lo que vieron — aunque no alcancen a corregirlo del todo.
- Si aplicaron reintento o cuarentena a alguna prueba, indicar cuál y por qué.

**Formato:** enlace al repositorio con el pipeline configurado y un archivo markdown con el reporte · **La entrega cierra al terminar las dos horas.** Cada integrante reporta en privado quién trabajó.



CIERRE

## Para la próxima semana

**La semana 16 no trae contenido nuevo:** es la semana de consolidación y cierre del curso.

- Se entrega el **Trabajo 2** (25 %): suite ejecutable, pipeline de CI, informe de métricas, registro de defectos y suite automatizada con patrones de diseño — todo lo trabajado desde la semana 9.
- Hay **socialización por equipos** del proyecto de Fábrica-Escuela, como cierre del semestre.

Revisen ahora el punto de control de la semana 13 y el estado de su pipeline: es la última semana con bloque de taller antes de la entrega final.

